

ESM Credit LOS — Application Lifecycle & State Machine

Source of truth: backend/src/credit/services/creditApplication.service.ts (the TRANSITIONS table, ~lines 178–248; transitionApplication() ~line 1083).

State enum: ApplicationState in backend/prisma/schema.prisma — imported from @prisma/client (do NOT re-create a local enum; credit.types.ts explicitly warns against it).

This document is the definitive map of how a credit application moves through its lifecycle, the permission each move requires, and the hard gates that can block a transition.

1. The 19 states

DRAFT, SUBMITTED, KYC_REVIEW, COMPLIANCE_HOLD, KYC_APPROVED, KYC_REJECTED, UNDERWRITING, CREDIT_ASSESSMENT, COMMITTEE_REVIEW, APPROVED, REJECTED, CONDITION_FULFILMENT, OFFER, ACCEPTED, DISBURSED, ACTIVE, CLOSED, WITHDRAWN, REFERRED_BACK

Terminal states (no outgoing transitions): REJECTED, CLOSED, WITHDRAWN.

2. State transition table

Each row: current state → action → next state. req = a **reason is required** for that action; ts = a timestamp field is set on transition.

Core forward path

From	Action	To	Notes
DRAFT	`submit`	SUBMITTED	ts `submittedAt`; blocked by **submission-readiness gate**
SUBMITTED	`start_kyc`	KYC_REVIEW	
KYC_REVIEW	`approve_kyc`	KYC_APPROVED	blocked if AML/PEP/sanctions adverse findings exist
KYC_REVIEW	`place_compliance_hold`	COMPLIANCE_HOLD	req
KYC_REVIEW	`reject_kyc`	KYC_REJECTED	req + rejection-reason-code
COMPLIANCE_HOLD	`clear_compliance_hold`	KYC_APPROVED	req
COMPLIANCE_HOLD	`reject_compliance`	KYC_REJECTED	req
KYC_APPROVED	`start_underwriting`	UNDERWRITING	
UNDERWRITING	`start_assessment`	CREDIT_ASSESSMENT	
CREDIT_ASSESSMENT	`submit_to_committee`	COMMITTEE_REVIEW	**heavily gated** (see §4)
COMMITTEE_REVIEW	`approve`	APPROVED	ts `decisionedAt`; **approval-chain gate**
COMMITTEE_REVIEW	`reject`	REJECTED	req + reason-code; ts `decisionedAt`; approval-chain gate

From	Action	To	Notes
APPROVED	`start_condition_fulfilment`	CONDITION_FULFILMENT	
CONDITION_FULFILMENT	`make_offer`	OFFER	**CP-fulfilment gate** (no unfulfilled precedent conditions)
APPROVED	`make_offer_direct`	OFFER	legacy direct path, still allowed
OFFER	`accept_offer`	ACCEPTED	**verified-LOO gate** + LOO-expiry gate
OFFER	`decline_offer`	REJECTED	req + reason-code; ts `decisionedAt`
ACCEPTED	`disburse`	DISBURSED	**blocked** — must go through DisbursementOrder workflow
DISBURSED	`activate`	ACTIVE	
ACTIVE	`close`	CLOSED	req; ts `closedAt`

Rejection / resubmission loops

From	Action	To	Notes
KYC_REJECTED	`resubmit`	SUBMITTED	loop back

Withdrawal (any non-terminal state)

withdraw → WITHDRAWN (req; ts closedAt) is allowed from: DRAFT, SUBMITTED, KYC_REVIEW, COMPLIANCE_HOLD, KYC_APPROVED, KYC_REJECTED, UNDERWRITING, CREDIT_ASSESSMENT, COMMITTEE_REVIEW, APPROVED, CONDITION_FULFILMENT, OFFER, ACCEPTED, REFERRED_BACK.

Refer-back / resume

From	Action	To
KYC_REVIEW / COMPLIANCE_HOLD / CREDIT_ASSESSMENT / COMMITTEE_REVIEW	`refer_back` (req)	REFERRED_BACK
REFERRED_BACK	`resume_kyc`	KYC_REVIEW
REFERRED_BACK	`resume_underwriting`	UNDERWRITING
REFERRED_BACK	`resume_assessment`	CREDIT_ASSESSMENT
REFERRED_BACK	`resume_committee`	COMMITTEE_REVIEW
REFERRED_BACK	`resubmit`	SUBMITTED

3. RBAC per action ([TRANSITION_PERMISSIONS](#))

Defined in `creditApplication.service.ts` (~line 261) and mirrored at route level in `creditApplication.routes.ts` via `requireTransitionPermission`. Action → required permission:

Permission	Actions
`credit:write`	submit, start_kyc, approve_kyc, resubmit, start_underwriting, start_assessment, submit_to_committee, accept_offer, withdraw, resume_*
`credit:approve`	place_compliance_hold, reject_kyc, clear_compliance_hold, reject_compliance, approve, reject, start_condition_fulfillment, make_offer, make_offer_direct, decline_offer, refer_back
`credit:disburse`	disburse (SOD separation from admin)
`credit:admin`	activate, close

> At the route layer, unknown actions **default to** `credit:approve` (fail-strict).

4. Hard gates in `transitionApplication()` (in execution order)

These are the guards that can block a transition with a specific error. When you add new business rules for a stage, this is where they belong.

On submit (**DRAFT**→**SUBMITTED**):

- `validateSubmissionReadiness(id, { stage: 'submission' })` — intake validation; blocks with the list of failing fields.

On approve_kyc:

- `getAmlAdverseFindings(id)` — if adverse AML/PEP/sanctions findings exist, forces "Place Compliance Hold" instead of approving.

On any `isCommitteeEntryAction` (**submit_to_committee, resume_committee**):

- 1 **Three-way CA-memo signoff** — `PREPARED_BY`, `REVIEWED_BY`, `CONCURRED_BY` must all be signed.
- 2 **Segregation of duties** — no two of those three signer roles may be the same user.
- 3 **Score-run required** — ≥ 1 `CreditScoreRun`; a manually-populated rating is insufficient.
- 4 **Score freshness** — latest run must not be stale vs. material inputs (latest `getLatestMaterialUpdate`).
- 5 **Absolute staleness ceiling** — default 30 days (`config.credit.scoreMaxAgeDays`, env `SCORE_MAX_AGE_DAYS`).
- 6 **SICR required** — corporate borrowers need ≥ 1 SICR assessment.
- 7 **Financial statements** — non-retail borrowers: balance sheets must have line items and balance (`Assets = Liabilities + Equity`, RM 1 tolerance).
- 8 `enforceCommitteeEntryGate` — run **last**; it validates readiness, then **freezes** the assessment result and **locks** the memo version. Both are irreversible, so nothing that can reject may run after this point (`committeeEntryGate.ts`).

On approve / reject (**COMMITTEE_REVIEW**→**terminal**):

- **Approval-chain gate** — all required approval decisions must have been collected via the approval-actions endpoint. Required count comes from `approvalMatrixService.lookupApprovalAuthority(totalExposure, borrowerRating, branchId, lane)`. Skipped when called from committee finalization (has its own quorum/voting) or admin bulk ops.

- **SOD check** — the recommendation author cannot be the final decision actor (`creditRecommendationService.checkSodSeparation`).

On activate / disburse:

- **Collateral valuation freshness** — hard-block if any tangible collateral valuation is > 12 months old (`collateralInsuranceMonitor.job`).

On make_offer (from CONDITION_FULFILMENT):

- **CP-fulfilment gate** — no PRECEDENT condition may be unfulfilled and not formally waived.

On accept_offer:

- **Verified-LOO gate** — a verified signed Letter of Offer must exist as a LETTER_OF_OFFER document with `verificationStatus=VERIFIED`.
- **LOO expiry** — `looservice.checkExpiry`; blocked if expired.

On disburse:

- **Disbursement-order gate** — direct ACCEPTED→DISBURSED is **blocked**; a `DisbursementOrder` must exist with `status=APPROVED`, and the actual transition is performed by `disburseOrder()` (not here).

5. What a successful transition does

- 1 Validates the transition against the TRANSITIONS table; throws `Invalid transition ... Valid actions: ... on a bad move.`
- 2 Applies the gates above.
- 3 In a `$transaction`:
 - `updateMany` with `{ id, state: existing.state }` guard — if 0 rows, throws **409 "Application state changed since read"** (race-safe, §F25).
 - Sets the transition's timestamp field.
 - Sets `rejectionReason/rejectionReasonCode` (reject/decline) or `withdrawalReason` (withdraw).
 - **Increments** version on every state change (§2.3).
 - Writes a `CreditAuditEvent` (`createAuditEvent`) with action, from→to state, reason.
- 1 **After** the transaction (failures non-blocking):
 - On submit: derives and sets the connected-party flag from `RelatedPartyGroup` membership.
 - Dispatches a credit notification event + webhook event to subscribers.
 - If the transition enters/leaves exposure states (APPROVED, OFFER, ACCEPTED, DISBURSED, ACTIVE), refreshes denormalised borrower exposure (`refreshBorrowerExposure`).

6. Related lifecycle state machines

- **Disbursement** is a separate workflow: `DisbursementOrder` (create → approve → disburse), governed by `disbursement.service.ts` and `disbursement.routes.ts` (`/applications/.../disbursement`). The application state DISBURSED is reached *through* this workflow, not directly.

- **Conditions** have their own status (`ConditionStatus`: PENDING/COMPLETED/WAIVED/EXPIRED) and type (`ConditionType`: PRECEDENT/SUBSEQUENT) in `condition.service.ts`.
- **LOO** has an expiry lifecycle and scheduler (`looExpiry.scheduler.ts`, `loo.service.ts`).

7. Sub-resource edit guards (`stateGuard.util.ts`)

Parent application state gates whether sub-resources can be edited/deleted:

- **Editable states:** DRAFT, KYC_REVIEW, COMPLIANCE_HOLD, KYC_APPROVED, UNDERWRITING, CREDIT_ASSESSMENT, COMMITTEE_REVIEW, OFFER.
- **Deletable states:** the above minus OFFER. **Deletion is blocked from APPROVED/OFFER/ACCEPTED onward** (LOS-007) — post-decision corrections must be a controlled supersession (upload a new version), never a delete, because documents are retained decision evidence.

`requireEditableState()` and `requireDeletableState()` throw 400 when violated.

8. Optimistic concurrency

`updateApplication(id, data, actorId, expectedVersion)` — pass the `expectedVersion` you read; a mismatch is rejected. Combined with the version increment on transitions, this prevents lost updates from concurrent edits.